AI-Powered Ground Gimbal Robot for Autonomous Video Tracking and Cinematic Capture

Siming Fu
Innovation AI Robotics Engineer Internship
Simulation-Only Implementation
ROS 2 Humble + Gazebo Fortress
August 2026

***Abstract*—This work presents a simulation-based autonomous cameraman robot that integrates AI person detection, active camera framing, mobile tracking, cinematic motion, and autonomous navigation in a modular ROS 2 stack. The platform uses a differential-drive mobile base, a two-axis yaw/pitch gimbal, a simulated RGB camera, 2D LiDAR, YOLOv8 perception, ros2_control, AMCL, and Navigation2. Four final operating modes were demonstrated: follow-behind tracking, side tracking, cinematic orbit filming, and autonomous indoor navigation. Quantitative validation was performed from ROS 2 bag recordings rather than visual inspection alone. During follow-behind motion, the subject distance averaged 2.177 ± 0.062 m with a yaw framing mean absolute error of 0.48°. The final cinematic orbit achieved a steady radius RMSE of 0.051 m relative to a 2.0 m command and covered 353.2° (98.1%) of a full revolution. Indoor navigation completed a 6.92 m route with a 0.169 m final position error and a minimum finite LiDAR range of 0.355 m during motion. A dedicated tracking-loss test produced a zero-velocity command 0.336 s after the last target update. The results demonstrate an integrated and measurable simulation prototype for autonomous ground-based filming.**

***Index Terms*—autonomous cameraman, ROS 2, Gazebo, YOLOv8, Navigation2, mobile robotics, gimbal control, person tracking, cinematic robotics**

I. Introduction

A. Problem Motivation

Autonomous filming systems combine perception, estimation, control, and navigation in a tightly coupled robotics problem. A useful cameraman robot must do more than detect a person: it must continuously interpret where the subject is relative to the camera, maintain a visually stable composition, coordinate camera orientation with base motion, avoid unsafe movement, and recover predictably when information is lost.

The internship project statement defined the target product as an AI-powered ground gimbal robot capable of following a person, maintaining framing, stabilizing video, navigating environments, and performing cinematic behaviors such as orbit and side tracking [1]. The project was executed entirely in simulation, so the engineering emphasis was placed on software architecture, controller behavior, ROS 2 integration, repeatability, and quantitative validation.

B. Final Project Scope

The final implemented scope includes:

- a differential-drive mobile base modeled in URDF/Xacro and simulated in Gazebo;
- a two-axis yaw/pitch camera gimbal controlled through ros2_control;
- a simulated RGB camera and 2D LiDAR;
- YOLOv8-based real-time person detection and temporal filtering;
- monocular target-distance and base-relative bearing estimation;
- independent visual framing and gimbal stabilization control;
- follow-behind mobile tracking with acceleration limiting and target-loss protection;
- an experimental side-tracking cinematic controller for a nonholonomic base;
- a map-frame 2.0 m cinematic orbit using Nav2 /navigate_through_poses;
- autonomous indoor navigation using map server, AMCL, Nav2 planner/controller, and velocity smoothing;
- automatic AMCL initial-pose initialization and an automatic indoor goal sender;
- RViz2 visualization, rosbag2 recording, and offline quantitative evaluation.

The implementation does **not** include physical hardware. Real motor dynamics, camera exposure artifacts, physical sensor calibration, embedded-compute latency, wheel slip, and real-world safety certification are therefore outside the scope of the measured results.

C. Original Requirements and Final Interpretation

The Week 1 architecture document treated several items as proposed baselines rather than fixed product requirements. In particular, it initially considered a three-axis gimbal, IMU integration, a generic behavior manager, and broader use of Nav2. As implementation progressed, the architecture was refined to match the actual system: a two-axis yaw/pitch gimbal was sufficient for the simulated tracking task; LiDAR, odometry, TF, and AMCL formed the final navigation localization chain; and mode-specific controllers replaced a monolithic behavior manager.

This evolution is consistent with the project statement, which explicitly allowed either a two-axis or three-axis gimbal during the stabilization phase and required final demonstration of person tracking, orbit filming, and indoor navigation [1].

II. Final System Architecture

A. Architecture Evolution

Figure 1 shows the Week 1 proposal. It was intentionally broad and included components planned for later evaluation. Figure 2 shows the final implemented structure used for the Week 8 demonstrations.

The architecture is intentionally modular. Perception, target-state estimation, gimbal framing, stabilization, cinematic behavior, navigation, and low-level actuation can be started, tested, and diagnosed separately. This proved important during integration because a visible failure in robot motion could originate from perception, TF, localization, Nav2 lifecycle state, command ownership, ros2_control, Gazebo state, or the controller itself.

B. Core Data Flow

The main perception-to-control flow is:

$$\begin{aligned}\text{RGB image} &\rightarrow \text{person detection}\\ &\rightarrow \text{framing error} \rightarrow \text{gimbal control},\\ &\rightarrow \text{target state} \rightarrow \text{base behavior}.\end{aligned}$$

The RGB camera publishes /camera/image_raw. The person_detector node runs YOLOv8 person detection and publishes smoothed angular/framing information together with a target-follow observation. The target_follow_state_estimator combines the camera observation with the measured gimbal yaw joint state to estimate the target bearing and distance relative to base_footprint. This state is published on /camera/target_follow_state and is consumed by the behavior controllers.

The visual framing branch is independent of base motion. The framing_gimbal_controller converts stable angular target error into a desired gimbal orientation, and gimbal_stabilizer applies the final rate-limited PID correction before sending joint targets through gimbal_position_controller.

C. Direct-Control Modes versus Nav2 Modes

A key final architecture decision is that not every behavior uses the same base-motion owner.

This separation prevents competing publishers from commanding the differential-drive base. Each scenario launch file enables exactly one intended motion owner.

D. Major ROS 2 Nodes and Interfaces

III. Robot and Simulation Platform

A. Mobile Base

The robot uses a differential-drive base with two powered wheels and low-friction fixed support casters. The final main geometry is summarized in Table III.

Differential drive was selected because it is simple to model, directly supported by ros2_control and Nav2, and sufficient for follow-behind, orbit, and indoor navigation. Its main limitation is the inability to command lateral velocity, which directly affects side tracking and is discussed in Section V-B.

B. Two-Axis Gimbal

The final camera gimbal uses yaw and pitch joints. The kinematic chain is:

$$\text{base} \rightarrow \text{gimbal mount} \rightarrow \text{yaw} \rightarrow \text{pitch} \rightarrow \text{camera}.$$

The yaw joint range was expanded during development to avoid artificial discontinuities during large cinematic rotations. The final simulated yaw limit supports multiple revolutions in either direction, while the controller applies a rate limit so that commanded position changes remain smooth. The gimbal position controller operates through ros2_control, and the stabilization layer uses a PID-based correction with bounded output.

A two-axis system was sufficient because the simulated ground platform does not require the same roll compensation as a flying camera platform. The original Week 1 three-axis concept was therefore simplified without compromising the primary tracking and cinematic objectives.

C. Sensors and Simulation Environment

The final robot uses:

- a simulated RGB camera publishing approximately 640×360 images;
- a 2D LiDAR used by Nav2 costmaps and indoor obstacle avoidance;
- wheel odometry from diff_drive_controller;
- TF/TF-static transforms describing robot kinematics and localization;
- AMCL pose estimation for map-frame navigation scenarios.

Gazebo Fortress [4] is connected to ROS 2 Humble [2] using ros_gz. RViz2 provides map, path, TF, robot-model, and camera visualization. The entire implementation runs in WSL2 Ubuntu 22.04 with ROS 2 Humble and Cyclone DDS.

IV. AI Perception and Gimbal Tracking

A. YOLOv8 Person Detection

The perception subsystem uses a YOLOv8n model [5] to detect the person class in each available simulated camera frame. The detector publishes an annotated image for visualization and a compact target observation containing horizontal image error, observed bounding-box height ratio, and detector confidence.

Temporal filtering is used to reduce frame-to-frame detector jitter before control. The control pipeline does not require the detector to estimate a full metric 3D pose. Instead, image-space measurements are transformed into the minimal target-relative quantities needed for framing and following.

B. Base-Relative Target State Estimation

The target_follow_state_estimator uses the horizontal camera error, bounding-box height ratio, confidence, and measured gimbal yaw. The final implementation defines the base-relative target bearing as

$$\theta_{target} = \operatorname{wrap}\left(\theta_{gimbal} - e_{camera}\right), \tag{1}$$

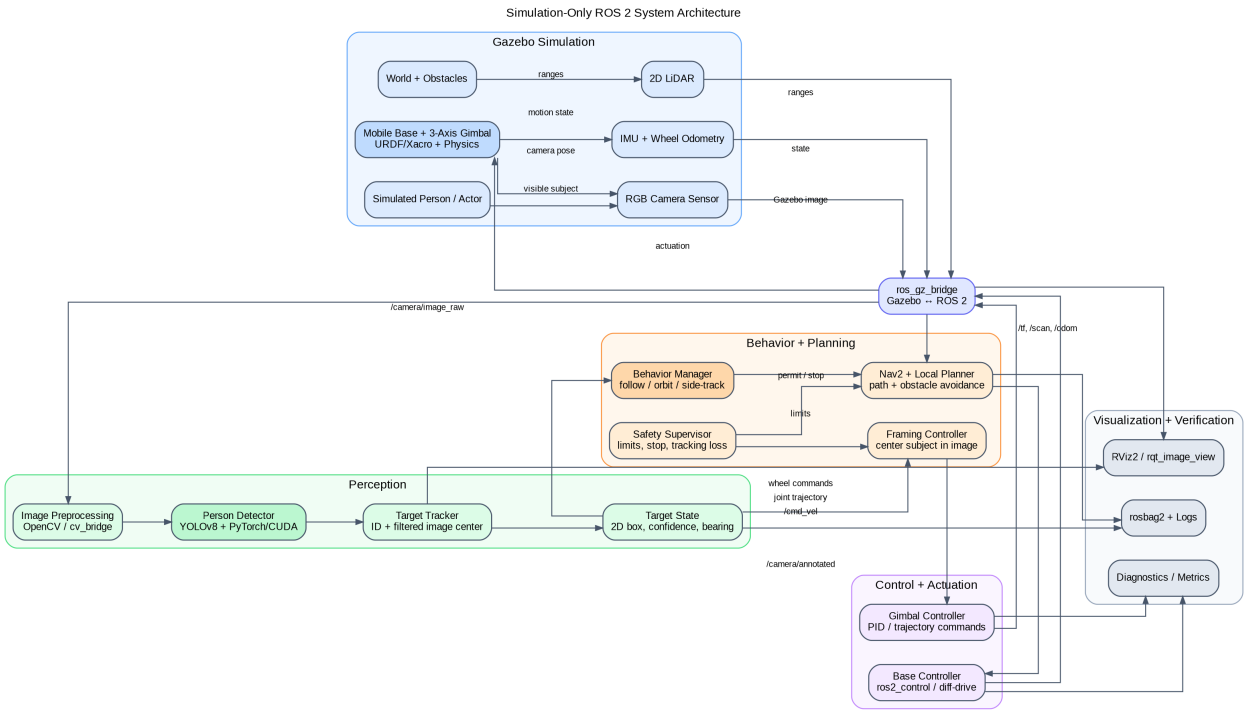


Fig. 1. Week 1 proposed architecture. The document explicitly treated this as an initial baseline subject to refinement during implementation.

TABLE I
BASE COMMAND OWNERSHIP BY FINAL SCENARIO.

Mode	Base motion owner	Reason
Follow-behind	person_follow_controller	Simple target-relative distance/bearing control benefits from direct low-latency velocity commands.
Side tracking	side_tracking_controller	The mode is a target-relative nonholonomic approximation and does not require a global path.
Cinematic orbit	Nav2 via cinematic_orbit_controller	The controller generates a global map-frame path and delegates path tracking to Nav2.
Indoor navigation	Nav2	Global planning, obstacle avoidance, AMCL localization, and behavior-tree execution are required.

TABLE II
PRINCIPAL IMPLEMENTED ROS 2 NODES.

Node	Primary input(s)	Primary output / responsibility
person_detector	/camera/image_raw	Person detection, confidence, framing/angular error, target-follow observation, annotated image.
target_follow_state_estimator	target observation, /joint_states	Base-relative bearing, distance, confidence on /camera/target_follow_state.
framing_gimbal_controller	stable angular target error	Desired yaw/pitch target on /gimbal/target_orientation.
gimbal_stabilizer	desired orientation, joint state	Rate-limited PID position commands for the gimbal.
person_follow_controller	target follow state	Follow-behind differential-drive command.
side_tracking_controller	target follow state, odometry	Experimental side-follow differential-drive command.
cinematic_orbit_controller	target follow state, TF	Subject map position, orbit path, Nav2 NavigateThroughPoses goals.
nav2_initial_pose_publisher	configured start pose	Repeated /initialpose publication for AMCL initialization.
indoor_demo_goal_sender	AMCL readiness	Automatic indoor NavigateToPose goal.

where e_{camera} is positive toward image-right while ROS/gimbal yaw is positive in the opposite angular direction. The wrap operation uses $\mathrm{atan2}(\sin\theta, \cos\theta)$.

Distance is estimated from the apparent bounding-box height ratio h_r using a one-point calibrated scale:

$$\hat{d} = \min\left(\frac{1.804}{h_r},\ 10.0\right)\ \text{m}. \tag{2}$$

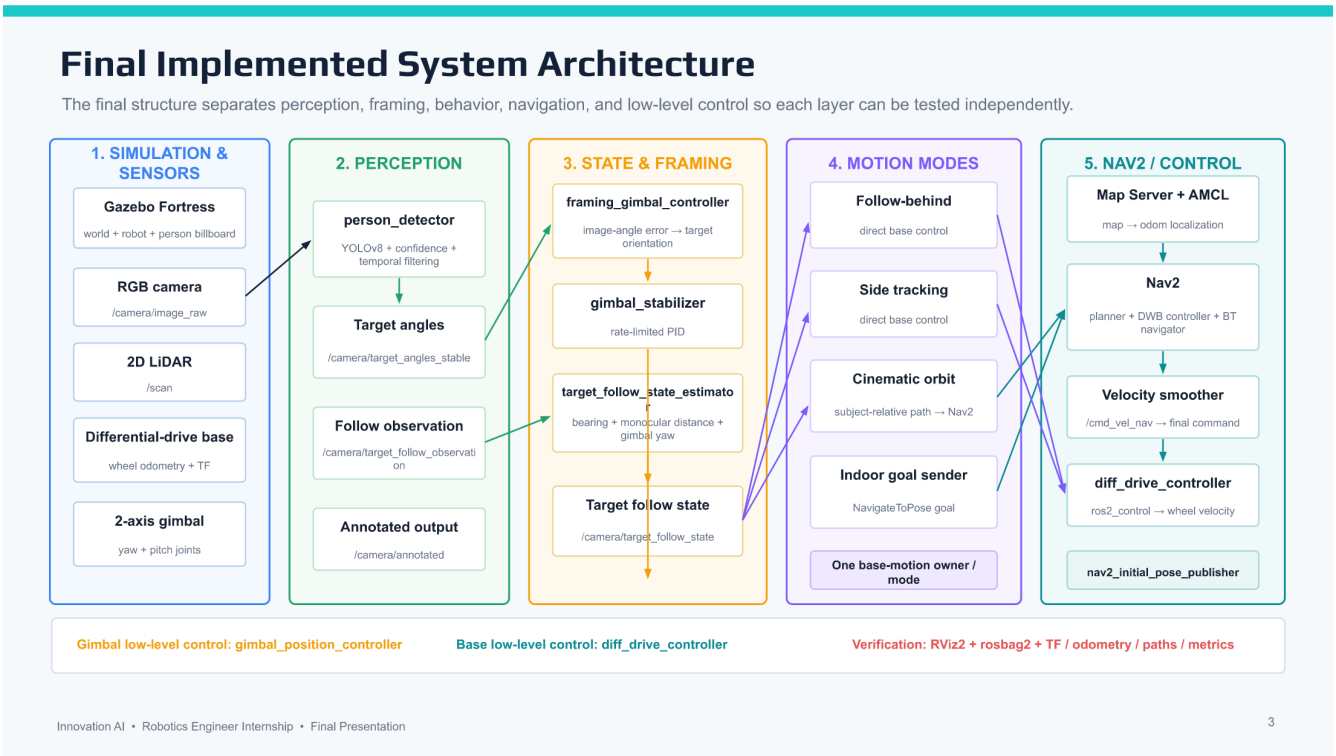


Fig. 2. Final implemented system architecture used for the integrated demonstrations.

TABLE III
FINAL SIMULATED ROBOT CONFIGURATION.

Parameter	Final value / configuration
Drive type	Differential drive
Base footprint	approximately 0.45×0.35 m
Base mass	8.0 kg
Wheel radius	0.08 m
Wheel separation	0.39 m
Wheel friction	$\mu_1 = \mu_2 = 1.0$
Support caster friction	$\mu_1 = \mu_2 = 0.01$
Base controller	diff_drive_controller
Odometry topic	/diff_drive_controller/odom

The scale 1.804 was calibrated from a known 2.5 m target distance and an observed height ratio of approximately 0.7216. This simple monocular estimate is intentionally lightweight but assumes a target with approximately fixed apparent human scale. Its limitations are discussed in Section VIII.

The final state message on /camera/target_follow_state stores bearing, distance, and confidence. Invalid, stale, or low-confidence observations are prevented from commanding continued autonomous motion.

C. Framing and Stabilization Pipeline

Gimbal framing is deliberately separated from base motion. The detector publishes a stable angular target error. The framing_gimbal_controller converts that error into a desired orientation, while gimbal_stabilizer uses measured joint state and PID correction to produce position commands.

This separation allows the gimbal to compensate for base rotation during follow, side, orbit, and indoor navigation. For example, the base is intentionally oriented tangentially during cinematic orbit rather than constantly pointed toward the subject. The gimbal therefore carries the primary responsibility for visual composition.

V. Autonomous Motion and Cinematic Modes

A. Follow-Behind Tracking

The follow-behind controller maintains a configurable target distance of 2.0 m while steering toward the detected target

bearing. The final implementation includes acceleration limiting, hysteresis around the desired distance, confidence gating, and a target-loss timeout. If the target is unavailable or stale, the controller publishes zero base velocity.

A separate moving-target controller changes the simulated billboard position during the scenario. This provides a repeatable walking-subject approximation while keeping the perception input visually realistic from the robot camera.

The direct-control architecture is appropriate for this mode because the behavior depends primarily on the current relative target state rather than a precomputed global path. Nav2 is therefore not inserted into the follow loop, avoiding unnecessary planning latency and command ownership complexity.

B. Side Tracking

Ideal side tracking resembles lateral translation while maintaining a camera-facing relationship to the target. A differential-drive base cannot directly command lateral velocity v_y, so true side translation is kinematically impossible.

The final side-tracking controller therefore implements a nonholonomic approximation. It chooses a preferred side, modifies the desired bearing as a function of distance error, rotates toward a feasible drive direction, and advances at a low tracking speed. The gimbal independently maintains subject framing. Linear and angular acceleration limits are applied so the controller does not produce abrupt commands.

The mode was intentionally kept conservative: the final test used a maximum observed linear command of 0.08 m/s and a maximum angular command of 0.45 rad/s. The resulting path is curved rather than laterally parallel. This is an engineering trade-off rather than a controller error; an omnidirectional or mecanum base would be the appropriate mechanical platform for true cinematic lateral tracking.

C. Cinematic Orbit

The cinematic orbit controller estimates the subject position in the map frame from the target-follow state and the current map to base_footprint transform. For the final static-subject demo, the subject center is locked when the orbit begins. Locking prevents small perception fluctuations from moving the center of the commanded circle during execution.

Given a locked subject position (x_s, y_s), radius r, and orbit angle θ_i, each orbit waypoint is generated by

$$x_i = x_s + r\cos\theta_i, \tag{3}$$

$$y_i = y_s + r\sin\theta_i. \tag{4}$$

The base orientation is tangential to the circle:

$$\psi_i = \theta_i + s\frac{\pi}{2}, \tag{5}$$

where $s = +1$ or -1 determines the orbit direction. The final nominal radius is 2.0 m.

The generated 25-pose closed representation contains the starting/closing geometry. For execution, the current-start and duplicate closing structure is handled so that 24 navigation targets are sent in two 12-waypoint phases. The split reduced repeated-goal reliability issues encountered when a long multi-pose action was executed as a single sequence.

Nav2 [3] receives the trajectory through /navigate_through_poses. Its DWB local controller tracks the path, while the gimbal continues to frame the subject. The orbit scenario uses a dedicated occupancy map matched to the clean orbit Gazebo world so that Nav2 does not plan around obsolete geometry from earlier test maps.

D. Autonomous Indoor Navigation

The indoor scenario demonstrates map-based navigation independently from target-relative following. A dedicated Gazebo world and matching occupancy map contain walls, a cabinet-like obstacle, and the stationary person billboard near the destination.

Localization uses map server and AMCL. The reusable nav2_initial_pose_publisher sends the configured initial pose repeatedly after localization startup, providing deterministic map $\rightarrow$ odom availability. The indoor_demo_goal_sender waits for localization output and sends a NavigateToPose goal near the person billboard. Nav2 then performs global planning, local path following, obstacle avoidance, and velocity smoothing.

The final automatic indoor goal is approximately $(4.465, -2.014)$ m in the map frame. The gimbal/perception pipeline remains active during navigation, allowing the camera to reacquire and frame the subject near the destination.

VI. Experimental Validation and Results

A. Evaluation Methodology

Four final rosbag2 recordings were captured, one for each operating mode. Together, the final bags contain 142,417 ROS messages. The recorded topics include target state, angular framing error, base commands, odometry, joint state, TF, AMCL pose, LiDAR, Nav2 plans, orbit path, and subject map position as appropriate for each scenario.

For stamped robot-state topics, analysis uses the message header timestamp rather than raw SQLite receive order. This is important because recorded receive timestamps can be interleaved by executor scheduling. Cross-topic event timing that involves unstamped command messages is aligned using recording timestamps or, for the dedicated tracking-loss test, the recorded /clock signal.

"Physical motion" duration is defined from the interval over which odometry reports non-negligible linear or angular velocity. This separates robot movement from launch initialization, target waiting time, and post-run observation time.

B. Follow-Behind Results

Figure 3 shows estimated subject distance during physical robot motion. The controller begins outside the tighter analysis band and quickly converges near the 2.0 m target. The final quantitative results are summarized in Table IV.

The mean distance is intentionally above exactly 2.0 m because the controller includes stable-distance behavior rather than aggressively forcing zero steady-state distance error. The

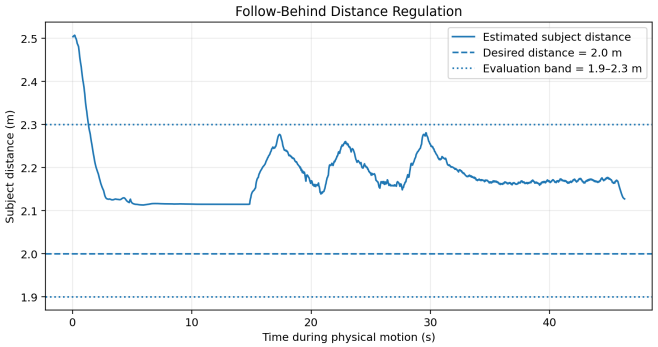


Fig. 3. Follow-behind estimated subject distance during the final physical-motion interval. The 1.9–2.3 m lines are a tighter post-test evaluation band; the original Week 1 acceptable range was broader.

TABLE IV
FINAL FOLLOW-BEHIND METRICS.

Metric	Result
Physical motion interval	46.30 s
Mean subject distance	2.177 ± 0.062 m
Final subject distance	2.127 m
Final error from 2.0 m setpoint	0.127 m
Distance RMSE from 2.0 m	0.188 m
Time within 1.9–2.3 m analysis band	97.15%
Yaw framing MAE	0.476°
Yaw framing 95th percentile	1.442°
Mean detection confidence	90.89%
Robot distance traveled	2.435 m
Maximum linear command	0.225 m/s
Target-state topic rate (simulation time)	29.57 Hz

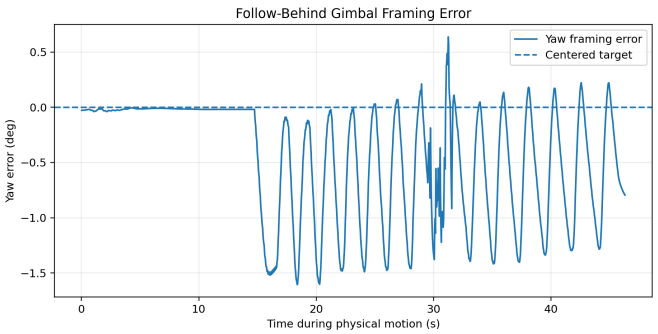


Fig. 4. Follow-behind yaw framing error. The independent gimbal controller keeps the target close to image center while the base regulates distance.

result is visually smooth and remains well inside the original 1.5–2.5 m Week 1 acceptance range. Figure 4 shows the yaw framing error during the same physical-motion interval.

C. Side-Tracking Results

Figure 5 shows the measured odometry trajectory. The curved path is expected from differential-drive kinematics. The robot traveled 3.786 m during a 57.12 s physical-motion interval.

The final 15 s of the moving evaluation interval were used as a steady-state framing window. Figure 6 shows the yaw

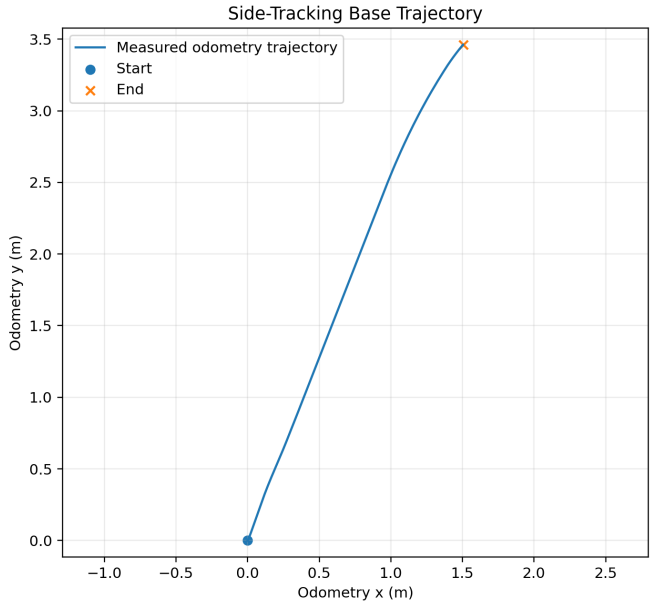


Fig. 5. Final side-tracking odometry trajectory. Curvature is an expected consequence of the nonholonomic differential-drive base.

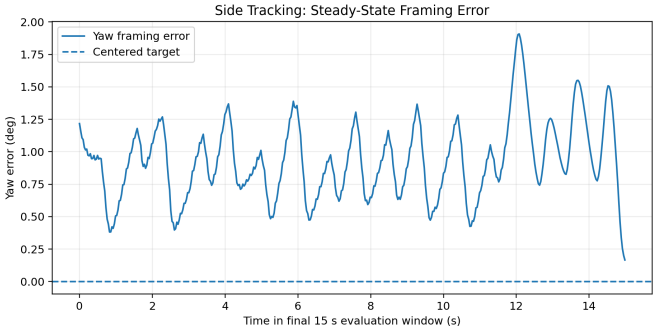


Fig. 6. Side-tracking yaw framing error during the final 15 s steady-state evaluation window.

TABLE V
FINAL SIDE-TRACKING METRICS.

Metric	Result
Physical motion interval	57.12 s
Robot distance traveled	3.786 m
Maximum linear command	0.080 m/s
Maximum angular command	0.450 rad/s
Final-15-s yaw framing MAE	0.928°
Final-15-s yaw framing 95th percentile	1.458°
Final recorded yaw framing error	0.118°
Mean confidence during physical motion	90.35%
Final base command	0.0 m/s, 0.0 rad/s
Target-state topic rate (simulation time)	29.48 Hz

error in that window.

These results demonstrate that side-view framing can be maintained despite the base's inability to strafe. However, the mode should be interpreted as an experimental cinematic

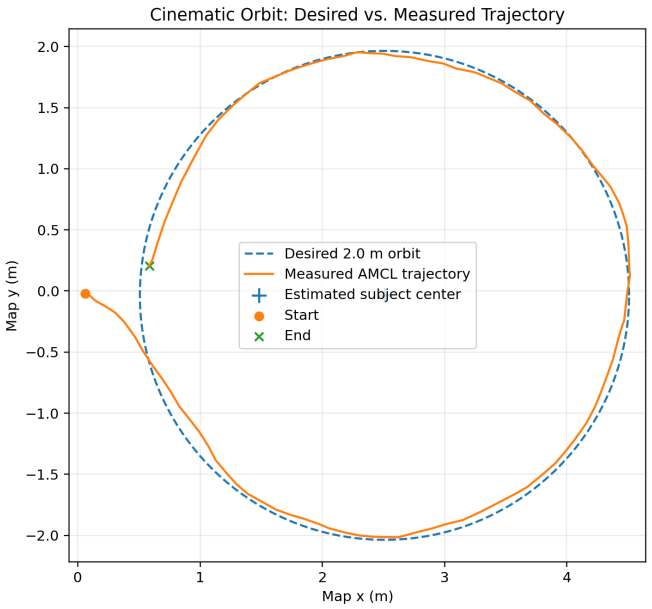


Fig. 7. Desired 2.0 m orbit and measured AMCL trajectory around the locked subject center.

approximation rather than a replacement for a holonomic tracking platform.

D. Cinematic Orbit Results

Figure 7 compares the requested 2.0 m circle with the AMCL-measured robot trajectory. The initial robot pose begins outside the orbit ring, so the first portion is an approach segment. Angular coverage is measured across the full moving trajectory around the locked center, while radius statistics are evaluated after the robot enters the orbit ring.

The estimated locked subject center was $(2.508, -0.036)$ m, giving a 0.0367 m error relative to the known billboard center at $(2.5, 0.0)$ m. The generated circular path itself is mathematically exact to floating-point precision: the mean planned radius is 2.000 m and the maximum numerical radius error in the recorded path is less than 5×10^{-16} m.

After entry onto the orbit ring, the measured radius was 1.967 ± 0.039 m. Figure 8 shows the radius trace.

The difference between 353.18° and exactly 360° should not be interpreted as a path-generation error. Nav2 declares waypoint completion using configurable pose tolerances; the robot center is not required to pass through the exact mathematical final coordinate. The planned circle itself closes exactly in the recorded orbit path.

E. Indoor Navigation Results

Figure 9 shows the AMCL-estimated trajectory from the initialized pose to the requested goal. The full recorded AMCL route length was 6.917 m.

The final AMCL pose was approximately $(4.314, -1.937)$ m, corresponding to a position error of 0.169 m relative to the requested goal $(4.465, -2.014)$ m. The physical-motion interval was 21.46 s. Figure 10 shows the nearest finite LiDAR return during that moving interval.

The minimum measured finite LiDAR range of 0.355 m is

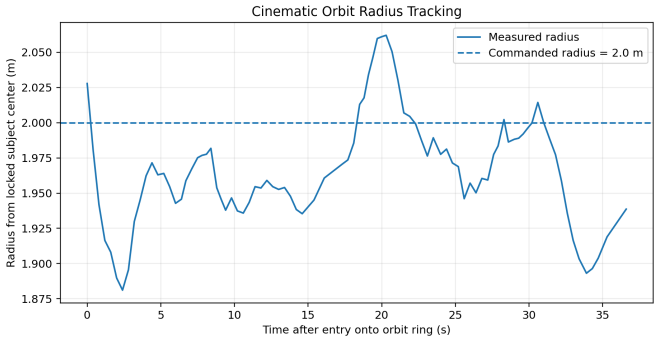


Fig. 8. Measured radius from the locked subject center after entry onto the orbit ring.

TABLE VI
FINAL CINEMATIC-ORBIT METRICS.

Metric	Result
Physical motion interval	39.22 s
Subject-center estimation error	0.0367 m
Commanded/planned orbit radius	2.000 m
Steady measured radius	1.967 ± 0.039 m
Steady radius RMSE	0.0511 m
Steady measured radius range	1.881–2.062 m
Angular traversal	353.18°
Fraction of full revolution	98.10%
Robot distance traveled	12.287 m
Maximum linear command	0.350 m/s
Maximum angular command	0.800 rad/s
Target-state topic rate (simulation time)	30.30 Hz

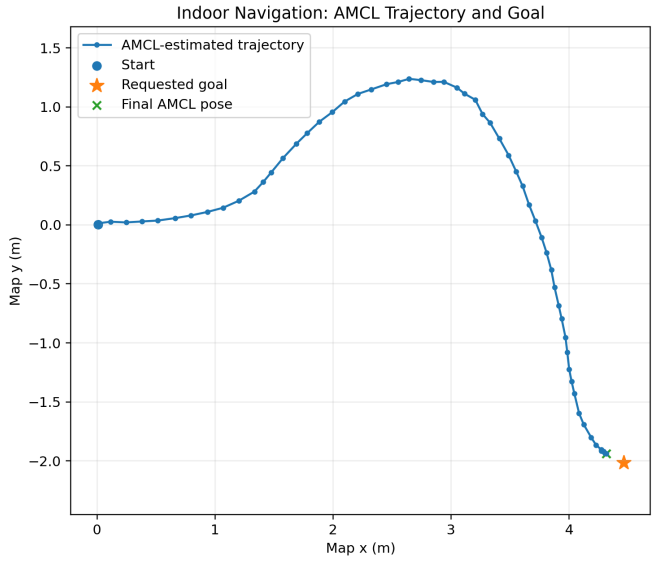


Fig. 9. Final indoor navigation AMCL trajectory, requested goal, and final localized pose.

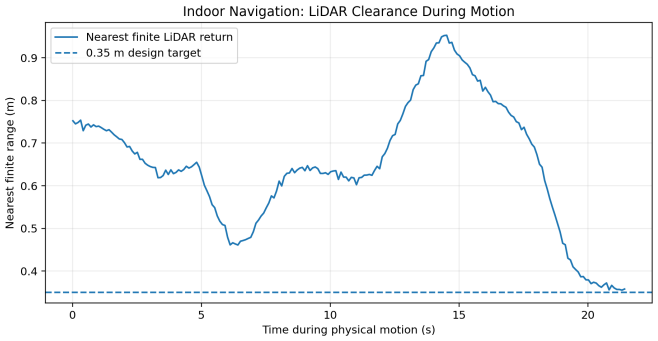


Fig. 10. Nearest finite LiDAR range during indoor physical motion. The dashed line marks the original 0.35 m Week 1 obstacle-clearance design target.

TABLE VII
FINAL INDOOR NAVIGATION METRICS.

Metric	Result
Physical motion interval	21.46 s
AMCL-estimated route length	6.917 m
Final position error	0.169 m
Minimum finite LiDAR range while moving	0.355 m
Median nearest finite LiDAR range while moving	0.641 m
Maximum linear command	0.350 m/s
Maximum angular command	0.560 rad/s
Target-state topic rate (simulation time)	28.92 Hz

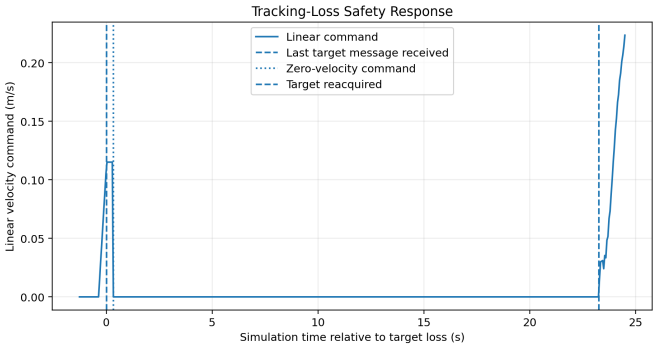


Fig. 11. Tracking-loss safety response. The controller transitions to zero velocity after target messages stop and resumes after target reacquisition.

consistent with the Week 1 proposed design target of at least 0.35 m clearance during autonomous motion. This value is a LiDAR range measurement rather than a formal footprint-to-obstacle clearance proof, so it should be interpreted as supporting evidence rather than a safety certification.

F. Tracking-Loss Safety Validation

A dedicated Week 7 test intentionally removed target updates while the robot was moving. The test recorded target state, command velocity, odometry, and simulation clock. Figure 11 shows the command response around the loss event.

The zero-velocity command was issued approximately 0.336 s after the last target message was received. Measured odometry became near-zero approximately 0.206 s after that stop command. After target reacquisition, the controller generated a new nonzero command approximately 0.035 s after receiving the recovered target message. This satisfies the original intent that target loss produce a bounded, prompt stop rather than continued blind motion.

G. System-Level Summary

VII. Engineering Challenges and Solutions

This project required significant integration debugging beyond implementing the individual algorithms. The most useful lessons came from failures where the visible symptom appeared in one subsystem but the root cause was elsewhere.

A. Challenge 1: Gazebo and ROS Runtime State

Problem. During one final orbit integration test, Nav2 successfully accepted the orbit goal and generated nonzero commands. The measured control pipeline showed approximately 0.35 m/s linear and -0.8 rad/s angular velocity at /cmd_vel_nav. The same command appeared at /diff_drive_controller/cmd_vel_unstamped, yet the robot did not move and the wheel-joint velocities remained essentially zero.

Investigation. Instead of changing DWB tuning or the orbit controller, the command path was checked layer by layer: orbit goal acceptance, Nav2 progress, /cmd_vel_nav, velocity-smoother output, diff-drive subscription ownership, controller lifecycle state, joint state, and Gazebo processes. The ROS graph revealed duplicated diff-drive subscriptions with the same node name, and Gazebo runtime state had become stale across repeated launches.

Solution. A complete WSL shutdown and restart removed the stale Gazebo/controller state. The unchanged controller code then operated normally. A clean-reset procedure became part of the integration workflow for severe runtime-state failures.

Lesson. In a distributed robotics system, a motion failure should not immediately be treated as a control-law failure. Observability across every interface is essential. The correct debugging question is often "where does the command stop being physically realized?" rather than "which gain should be changed?"

B. Challenge 2: Gimbal Yaw Wrap-Around and Large Rotation

Problem. Large base rotations and orbit maneuvers exposed discontinuities near the conventional $\pm\pi$ yaw representation. Commands that are mathematically equivalent modulo 2π can still produce undesirable controller behavior if a position-controlled joint interprets them as large jumps across a seam.

Solution. The simulated yaw joint range was expanded substantially, and the final controller uses rate-limited incremental position commands rather than unrestricted absolute jumps. The stabilizer PID was tuned after this representation issue was addressed, rather than attempting to hide the discontinuity through gain changes.

Lesson. Angular topology matters. Normalizing a measurement to $[-\pi, \pi]$ is useful for error computation, but physical joint-command continuity may require a wider unwrapped representation.

TABLE VIII
CONDENSED FINAL VALIDATION SUMMARY.

Capability	Primary objective	Final measured result
Follow distance	Maintain 2.0 m subject distance	2.177 ± 0.062 m; 0.127 m final error
Follow framing	Keep subject centered	0.476° yaw MAE
Side tracking	Maintain cinematic side view	0.928° steady yaw MAE
Orbit geometry	Maintain 2.0 m orbit	0.051 m steady radius RMSE
Orbit completion	Near-full circular capture	353.18° / 98.10%
Indoor navigation	Reach autonomous map goal	0.169 m final position error
Indoor sensing	Preserve obstacle margin	0.355 m minimum finite LiDAR range while moving
Target loss	Stop safely on missing target	0.336 s stop-command response

C. Challenge 3: Reliable Nav2 Orbit Integration

Orbit filming exposed several independent integration issues over time.

Map-frame target estimation. The initial target-to-map conversion was incorrect, producing a subject position almost one meter away from the known simulated target. Correcting the base-relative bearing interpretation reduced the target-center error to a few centimeters.

Repeated orbit execution. A long NavigateThroughPoses sequence could be accepted yet later cancel during repeated operation. Splitting the final 24 navigation targets into two 12-waypoint phases improved reliability and gave the orbit controller explicit phase-level state.

Localization initialization. Nav2 requires a valid map $\rightarrow$ odom $\rightarrow$ base_footprint chain. Manual RViz "2D Pose Estimate" was unsuitable for a repeatable final demo, so a reusable nav2_initial_pose_publisher was created. Startup timing initially depended on simulation time in a way that could prevent publication. The final implementation uses deterministic steady/wall-time scheduling for the initialization timer while preserving ROS simulation time in message headers.

Map/world consistency. The clean orbit Gazebo world no longer contained an old center obstacle, but the reused occupancy map still did. RViz therefore displayed a black obstacle that did not exist physically in Gazebo and Nav2 could treat it as mapped geometry. A dedicated orbit occupancy map was generated to match the final world.

Lesson. Navigation reliability depends on more than planner/controller tuning. TF availability, lifecycle activation, initial localization, world/map consistency, action semantics, and goal tolerances must all agree.

D. Challenge 4: Side Tracking with Differential Drive

Problem. A cinematic side-follow request implies lateral motion, but differential drive has no independent lateral degree of freedom. Early behavior therefore naturally curved and could lose the desired relationship during transients.

Solution. The controller was redesigned as a nonholonomic approximation. It uses a low forward speed, bearing correction, distance-dependent desired bearing, angular limiting, acceleration limiting, target confidence, and odometry-based motion logic. The gimbal carries much of the visual framing burden while the base maintains a feasible curved trajectory.

Lesson. Software cannot remove a mechanical kinematic constraint. The correct solution is to define an achievable behavior for the chosen platform and document the trade-off. If true lateral cinematography were a primary product requirement, an omnidirectional chassis would be a better architecture.

E. Challenge 5: Tracking-Loss Safety and Recovery

Problem. An autonomous person-following robot must not continue driving on stale perception after the person is lost.

Solution. Target messages are timestamped and confidence-gated. The base controller enforces a target timeout and publishes zero velocity when the target is no longer fresh. A dedicated loss/reacquisition experiment validated stop and recovery timing rather than assuming the logic worked from code inspection.

Lesson. Safety behavior should be measurable and testable as a state transition. Recording the actual time between last valid target information, stop command, physical settling, and reacquisition provides more useful evidence than a simple "target lost" log message.

VIII. LIMITATIONS AND ENGINEERING TRADE-OFFS

A. Simulation-Only Validation

All results in this report come from Gazebo simulation. The software architecture is designed to resemble a real ROS 2 robot, but simulation does not fully capture motor backlash, real encoder noise, imperfect tire-ground contact, camera motion blur, exposure changes, LiDAR multipath, embedded GPU scheduling, wireless-network variability, thermal limits, or physical emergency-stop requirements. Quantitative results should therefore be interpreted as simulation performance, not guaranteed hardware performance.

B. Billboard Target Model

The primary target is a textured person billboard rather than a fully articulated simulated human. This provides repeatable visual detection and a controlled geometric target but does not reproduce changing human pose, self-occlusion, limb motion, or body-scale variation. A realistic animated actor would be a stronger perception stress test.

C. Monocular Distance Estimation

Equation 2 assumes a fixed relationship between apparent person height and distance. It is sensitive to person scale, posture, cropping, camera pitch, and detector bounding-box variation. The method was sufficient for the controlled simulated target but would need replacement or fusion with depth sensing for a robust physical system.

D. Differential-Drive Cinematic Constraints

Differential drive was a strong choice for system integration and Nav2 compatibility, but it limits side tracking. A mecanum or other omnidirectional platform would support lateral motion directly, at the cost of more complex wheel modeling, controller tuning, and potentially less efficient straight-line motion.

E. Locked Orbit Center

The most reliable final orbit mode locks the subject center when the orbit starts. This is appropriate for a stationary or slowly moving filming subject, but it does not continuously re-center the path around a dynamically moving person. A future system should combine target-motion prediction with receding-horizon orbit generation or a dynamic local planner.

F. Topic Rate versus Wall-Clock Inference Throughput

The approximately 29–30 Hz target-state rates reported in Section VI are measured against simulation-message timestamps. They should not be interpreted as wall-clock YOLO inference throughput. Gazebo real-time factor and GPU processing load can cause simulation time and wall time to progress at different rates.

IX. Conclusions and Future Work

A. Final Outcome

The internship produced an integrated simulation-based autonomous cameraman platform with the major capabilities required by the project statement [1]:

- AI-powered human detection from a robot-mounted camera;
- two-axis stabilized yaw/pitch framing;
- target-relative follow-behind behavior;
- an experimental side-tracking cinematic mode;
- near-360-degree 2.0 m cinematic orbit filming;
- AMCL/Nav2-based autonomous indoor navigation and obstacle avoidance;
- tracking-loss stop and recovery behavior;
- repeatable mode-specific bringup launch files;
- quantitative validation using RViz2, rosbag2, odometry, AMCL, and LiDAR data.

The final results show that the system is not only visually demonstrable but measurable. The follow controller maintained approximately 2.18 m mean target distance, the orbit tracked a 2.0 m radius with approximately 5.1 cm RMSE after ring entry, indoor navigation finished within 0.17 m of the target goal, and target-loss handling generated a bounded stop response.

B. Engineering Lessons

Three broader engineering lessons were especially important.

First, modularity is a practical debugging tool rather than only a software-design preference. Separating perception, target estimation, gimbal framing, behavior control, Nav2, and ros2_control made it possible to identify the actual layer responsible for failures.

Second, quantitative logging changes the quality of engineering decisions. rosbag2 data made it possible to distinguish visual impressions from measured performance, compare desired and actual trajectories, and evaluate safety timing.

Third, mechanical architecture constrains software behavior. Differential drive is well matched to following, orbiting, and indoor navigation, but side tracking reveals a genuine nonholonomic limitation that cannot be solved by tuning alone.

C. Recommended Future Work

The highest-value next steps are:

1) replace the billboard with a realistic animated or physics-driven human model;
2) improve target range estimation using stereo/depth information or learned depth;
3) add Kalman filtering or motion prediction for dynamic subjects;
4) implement dynamic-obstacle tests and recovery behaviors in indoor navigation;
5) investigate an omnidirectional base for true lateral cinematic tracking;
6) support a moving-center orbit using receding-horizon replanning;
7) measure true wall-clock image-to-actuation latency end to end;
8) transfer the architecture to physical hardware and re-tune control gains under real sensor/actuator noise.

The project therefore provides both a completed simulation demonstration and a modular foundation for a future physical autonomous camera robot.

APPENDIX A
FINAL SCENARIO LAUNCH COMMANDS

The final integrated scenarios are launched through dedicated wrappers:

```
ros2 launch cameraman_robot_bringup week7_follow_behind_demo.launch.py
ros2 launch cameraman_robot_bringup week7_side_tracking_demo.launch.py
ros2 launch cameraman_robot_bringup week7_orbit_demo.launch.py
ros2 launch cameraman_robot_bringup week7_indoor_navigation_demo.launch.py
```

The orbit is started explicitly after localization and target state are available:

```
ros2 service call /cinematic/start_orbit std_srvs/srv/Trigger "{}"
```

APPENDIX B
KEY FINAL PARAMETERS

TABLE IX: Selected final system parameters.

Subsystem	**Parameter**	**Value / note**
Mobile base	wheel radius	0.08 m
Mobile base	wheel separation	0.39 m
Follow-behind	desired distance	2.0 m
Target estimator	distance scale	1.804
Target estimator	maximum distance	10.0 m
Side tracking	desired distance	2.0 m
Side tracking	tracking speed	0.08 m/s
Side tracking	observed max angular command	0.45 rad/s in final run
Cinematic orbit	nominal radius	2.0 m
Cinematic orbit	final navigation targets	24, split 12 + 12
Localization	AMCL update minimum distance	0.10 m
Localization	AMCL update minimum angle	0.10 rad
Gimbal	axes	yaw + pitch
Gimbal	control interface	position-controlled ros2_control joints

APPENDIX C
FINAL ROS 2 INTERFACE SUMMARY

TABLE X: Selected final ROS 2 topics.

Topic	**Message type**	**Purpose**
/camera/image_raw	sensor_msgs/msg/Image	Simulated RGB camera input
/camera/target_follow_observation	geometry_msgs/msg/Vector3Stamped	Camera horizontal error, height ratio, confidence
/camera/target_follow_state	geometry_msgs/msg/Vector3Stamped	Bearing, distance, confidence in base frame
/camera/target_angles_stable	geometry_msgs/msg/Vector3Stamped	Smoothed gimbal angular framing error
/gimbal/target_orientation	geometry_msgs/msg/Vector3	Desired yaw/pitch target
/gimbal_position_controller/commands	std_msgs/msg/Float64MultiArray	Low-level gimbal joint targets
/diff_drive_controller/cmd_vel_unstamped	geometry_msgs/msg/Twist	Final base command
/diff_drive_controller/odom	nav_msgs/msg/Odometry	Wheel odometry
/scan	sensor_msgs/msg/LaserScan	2D LiDAR ranges
/amcl_pose	geometry_msgs/msg/PoseWithCovarianceStamped	Map-frame localization
/cinematic/subject_position_map	geometry_msgs/msg/PointStamped	Orbit center estimate
/cinematic/orbit_path	geometry_msgs/msg/PoseArray	Generated circular orbit geometry
/cmd_vel_nav	geometry_msgs/msg/Twist	Nav2 controller output before final smoothing/remap

Topic	Message type	Purpose
/tf, /tf_static	tf2_msgs/msg/TFMessage	Coordinate transforms

APPENDIX D
VALIDATION DATA AND REPRODUCIBILITY

The accompanying LaTeX project includes CSV files derived from the recorded bags in the data/ directory and a standalone analysis script in scripts/generate_report_assets.py. The script performs minimal CDR decoding for the standard ROS 2 message types used by the final experiments, computes the report metrics, and regenerates all plots without requiring ROS 2 to be installed on the analysis machine.

The included files are intended to make the reported numbers traceable to recorded data rather than manually entered screenshots. The original rosbag SQLite3 files should be preserved separately as the authoritative raw experiment records.

REFERENCES

[1] Innovation AI, *8-Week Robotics Engineer Internship Project Statement: AI-Powered Ground Gimbal Robot for Autonomous Video Tracking & 360° Cinematic Capture*, 2026.

[2] Open Robotics, *ROS 2 Humble Documentation*. https://docs.ros.org/en/humble/

[3] Open Navigation LLC / ROS 2 Navigation Working Group, *Navigation2 Documentation*. https://navigation.ros.org/

[4] Open Robotics, *Gazebo Fortress Documentation*. https://gazebosim.org/docs/fortress/getstarted/

[5] Ultralytics, *Ultralytics YOLO Documentation*. https://docs.ultralytics.com/